

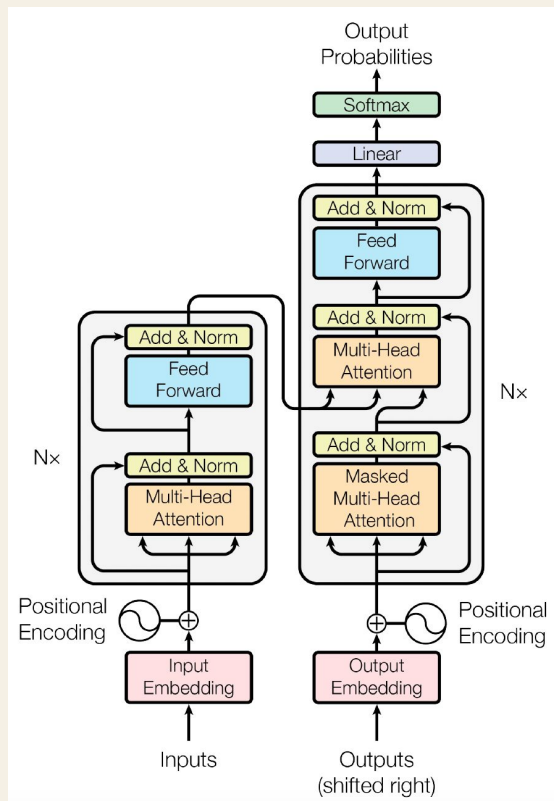
From Transformer to LLM

Transformer → GPT-2 → LLaMA-2 → Qwen-3

Why This Evolution Matters

- Modern AI systems are built on the Transformer architecture.
- Yet GPT-2, LLaMA-2, and Qwen-3 behave dramatically differently.
- What changed in the architecture?
- And what surprisingly stayed the same?
- Disclaimer: The components listed for each LLM indicate that the model *uses* these techniques; this does not imply that the model *first proposed* them.





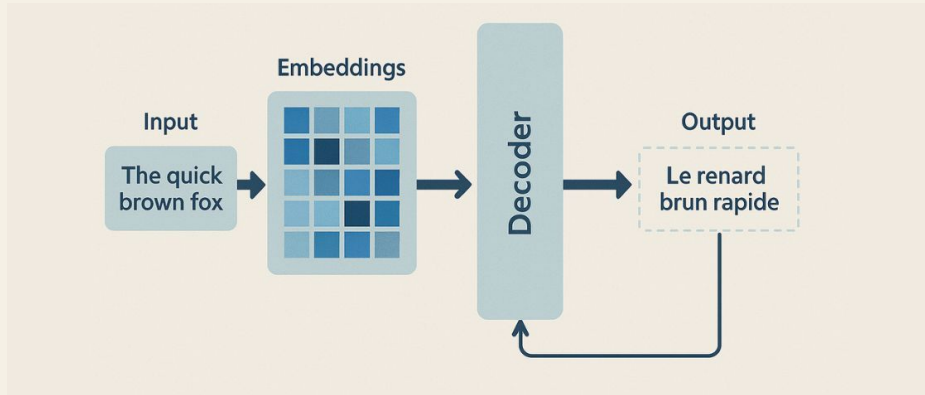
Part I

Transformer

(Google, 2017)

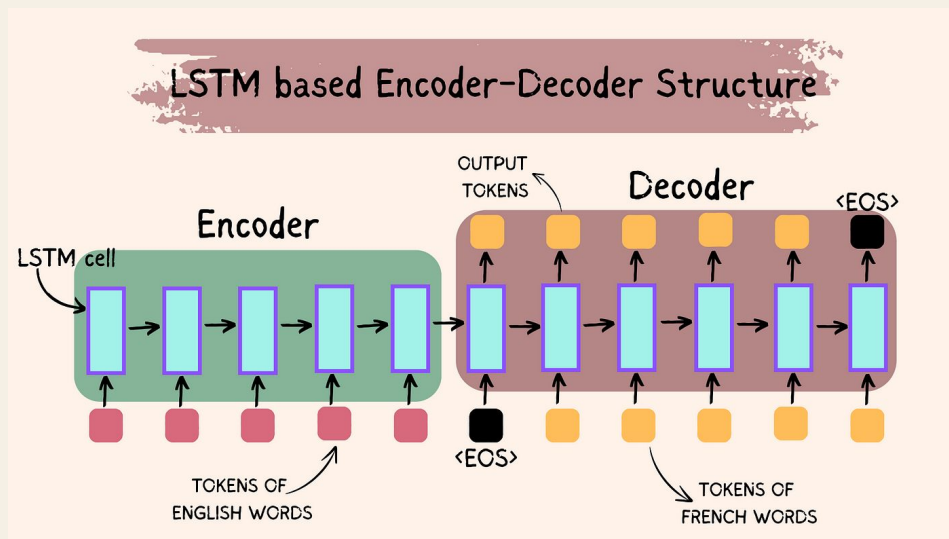
Recap of Transformer

- Transformer is a neural architecture introduced for machine translation, replacing recurrent models with self-attention.
- It enables parallel sequence modeling and captures global context efficiently.
- The same architecture now underlies modern large language models.



Background: Seq2Seq

- In seq2seq tasks like machine translation, the output length is not fixed.
- Unlike classification or segmentation, the output size is unknown beforehand.
- An <EOS> token signals the model when to stop generating.



RNN vs. Transformers

RNN vs Transformers

d2l.ai

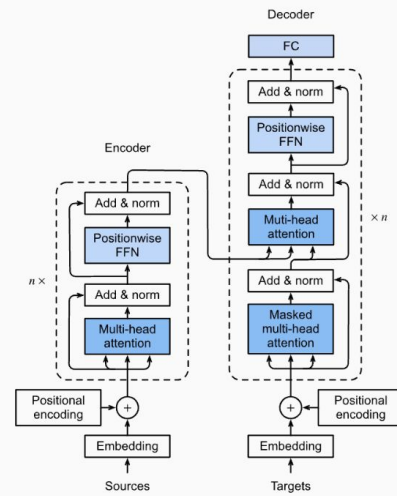
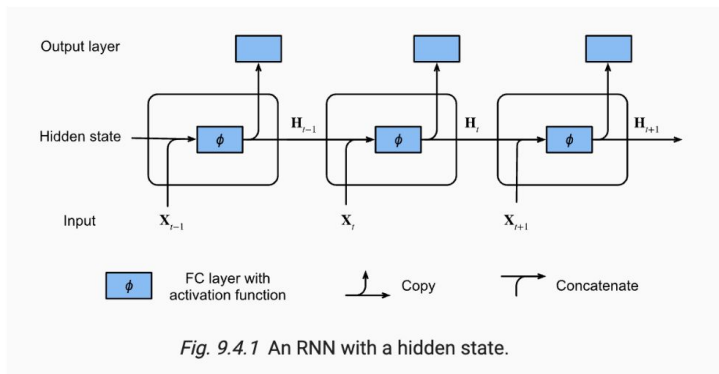


Fig. 11.7.1 The Transformer architecture.

Transformer Block

- Introduces self-attention as the main computation
- Uses residual + LayerNorm to stabilize training
- Uses position-wise feedforward network
- Complexity: $O(n^2)$ attention over sequence length



```
def transformer_layer(x):  
    # self-attention  
    attn_out = multi_head_attention(x)  
    x = layernorm(x + attn_out)  
  
    # feedforward  
    ff_out = feedforward(x)  
    x = layernorm(x + ff_out)  
  
    return x
```

LayerNorm

- It computes the mean and variance across the specified last dimensions for each sample.
- It normalizes the input using these statistics with a small epsilon for numerical stability.
- It then applies learnable scale (gamma) and shift (beta) parameters.

```
class MyLayerNorm(nn.Module):
    def __init__(self, normalized_shape, eps=1e-5):
        super().__init__()

        if isinstance(normalized_shape, int):
            normalized_shape = (normalized_shape,)

        self.normalized_shape = normalized_shape
        self.eps = eps

        # Learnable parameters
        self.gamma = nn.Parameter(torch.ones(*normalized_shape))
        self.beta = nn.Parameter(torch.zeros(*normalized_shape))

    def forward(self, x):
        # Compute mean and variance over last dimensions
        dims = tuple(range(-len(self.normalized_shape), 0))

        mean = x.mean(dim=dims, keepdim=True)
        var = x.var(dim=dims, keepdim=True, unbiased=False)

        x_norm = (x - mean) / torch.sqrt(var + self.eps)
        return self.gamma * x_norm + self.beta
```

Scaled Dot-Product Attention

- Attention = weighted sum of values
- Weights computed by query-key similarity
- Softmax makes it a probability distribution



```
def attention(q, k, v):  
    scores = (q @ k.transpose(-2, -1)) / sqrt(d_k)  
    weights = softmax(scores, dim=-1)  
    return weights @ v
```

Attention lets each token directly attend to every other token.

Multi-Head Attention

- Project input into multiple subspaces
- Each head learns different interaction patterns
- Heads are concatenated + projected back

```
def multi_head_attention(x):  
    q = Wq(x)  
    k = Wk(x)  
    v = Wv(x)  
  
    q = split_heads(q)  
    k = split_heads(k)  
    v = split_heads(v)  
  
    out = attention(q, k,  
v) out =  
combine_heads(out)  
    return Wo(out)
```

Sinusoidal Positional Encoding

- Self-attention is order-agnostic
- Position information added to token embeddings
- Fixed, non-learned sinusoidal functions
- Enables extrapolation to longer sequences



```
def sinusoidal_positional_encoding(pos, d_model):  
    pe = zeros(d_model)  
    for i in range(0, d_model, 2):  
        pe[i] = sin(pos / (10000 ** (i /  
d_model)))  
        pe[i+1] = cos(pos / (10000 ** (i /  
d_model)))  
    return pe
```

Implementation of Transformer Decoder (1)

```
class TransformerLM(nn.Module):
    def __init__(self, cfg: TransformerConfig):
        super().__init__()
        self.cfg = cfg

        self.tok_emb = nn.Embedding(cfg.vocab_size, cfg.d_model)
        self.pos_enc = SinusoidalPositionalEncoding(cfg.d_model, max_len=cfg.max_len)
        self.drop = nn.Dropout(cfg.dropout)

        self.blocks = nn.ModuleList(
            [TransformerBlock(cfg.d_model, cfg.n_heads, cfg.d_ff, dropout=cfg.dropout)
             for _ in range(cfg.n_layers)]
        )

        self.ln_f = nn.LayerNorm(cfg.d_model)
        self.lm_head = nn.Linear(cfg.d_model, cfg.vocab_size, bias=False)
```

Implementation of Transformers Decoder (2)



```
def forward(self, input_ids: torch.Tensor) -> torch.Tensor:
    """
    input_ids: (B, T)
    returns logits: (B, T, vocab_size)
    """
    B, T = input_ids.shape
    assert T <= self.cfg.max_len, "Sequence length exceeds max_len"

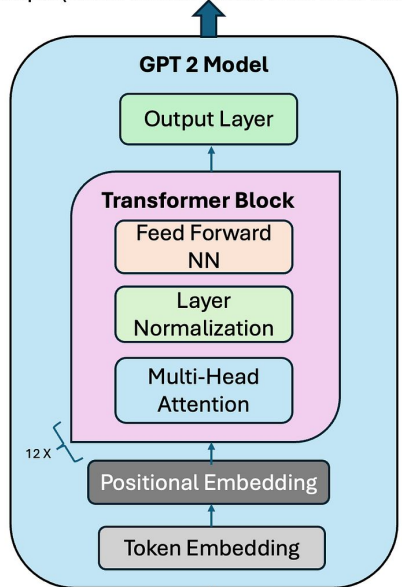
    x = self.tok_emb(input_ids)           # (B, T, D)
    x = self.pos_enc(x)                   # add sinusoidal positions
    x = self.drop(x)

    mask = causal_mask(T, device=input_ids.device) # (T, T)

    for block in self.blocks:
        x = block(x, attn_mask=mask)

    x = self.ln_f(x)
    logits = self.lm_head(x)
    return logits
```

Output(Same dimension as tokenized text)

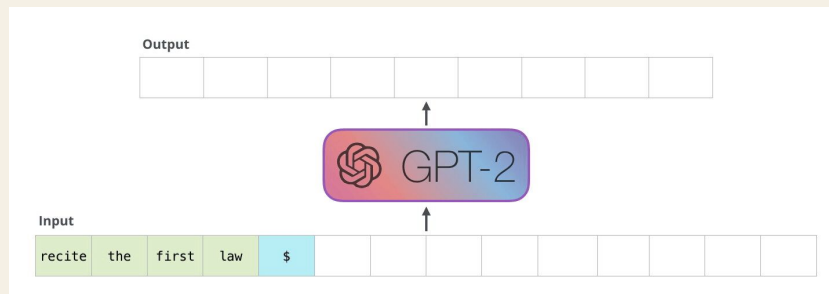


Part II: GPT-2

(OpenAI, 2019)

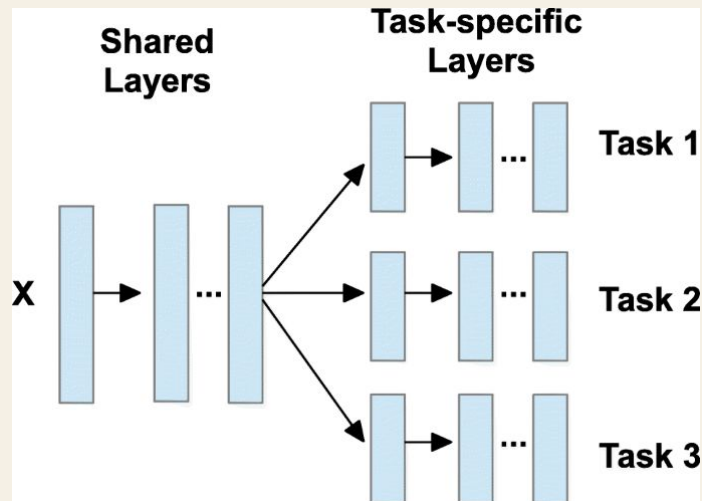
Overview of GPT-2

- GPT-2 is a decoder-only Transformer trained with autoregressive next-token prediction on large-scale web text.
- It shows that scaling a simple architecture yields strong zero-shot language understanding and generation.
- GPT-2 established the modern LLM paradigm: prompt-in, text-out.



From Seq2Seq to Multitask Learners

- Originally, Transformers were trained for specific tasks such as machine translation.
- In traditional multitask learning, models share a backbone while using task-specific heads.
- Next-token prediction on a large and diverse corpus can be viewed as implicit multitask learning across many tasks and domains.



Sinusoidal PE → Learned Positional Embeddings

- Transformer (2017) injects position using a fixed sinusoidal function.
- GPT-2 learns a trainable positional embedding table, optimized during training.
- This simplifies implementation but limits extrapolation beyond trained context length.



```
x = token_embed[token_ids] + pos_embed[position_ids]
```

Post-LayerNorm $\rightarrow$ Pre-LayerNorm

- The original Transformer applies LayerNorm after residual connections (Post-LN).
- GPT-2 applies LayerNorm before attention and MLP blocks (Pre-LN).
- This improves gradient flow and makes deep Transformers easier to train: In Pre-LN, the residual path is an identity mapping.



```
x = x + self_attn(layer_norm(x))  
x = x + mlp(layer_norm(x))
```

KV Cache for Fast Inference

- GPT-2 inference is optimized for autoregressive decoding by caching past keys and values.
- This avoids recomputing attention over the full prefix at every step.
- KV caching is essential for making long generation efficient in practice.



```
# at timestep t
q_t = Wq(x_t)           # query for current token
k_t = Wk(x_t)           # key for current token
v_t = Wv(x_t)           # value for current token

# append to cache
K[:, :, t] = k_t
V[:, :, t] = v_t

# attend to all previous tokens
attn = softmax(q_t @ K[:, :, :t+1].transpose(-2, -1))
out  = attn @ V[:, :, :t+1]
```

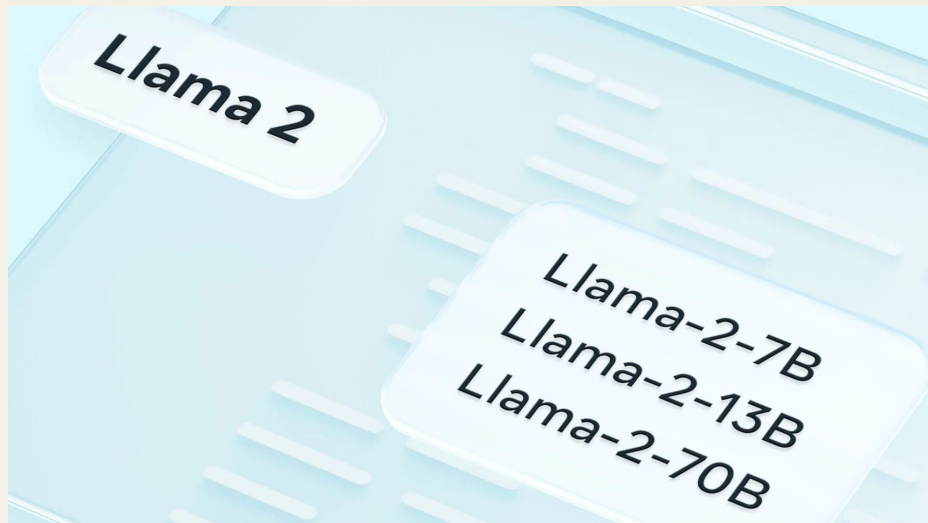


Part III: LLaMA-2

(Meta, 2023)

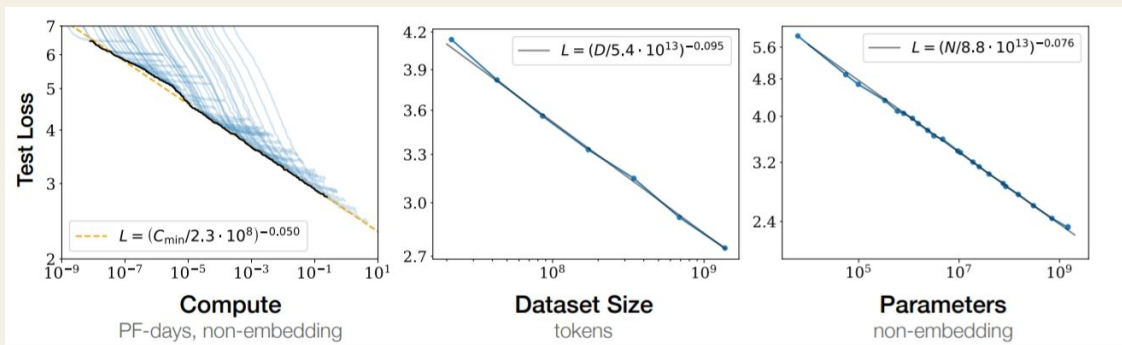
Overview of LLaMA-2

- LLaMA keeps the GPT-style decoder-only Transformer, but modernizes key components for stability and efficiency.
- Its core upgrades are RoPE positional encoding, RMSNorm, and a gated MLP (SwiGLU).
- These changes improve long-context behavior and quality per parameter without changing the attention math.



Scaling Law

- GPT-2: 1.5B parameters, ~40B tokens in training data
- LLaMA-2: 70B parameters, ~2T tokens in training data
- Chinchilla scaling-law: model performance improves predictably as a power-law with increased parameters, training data, and compute.



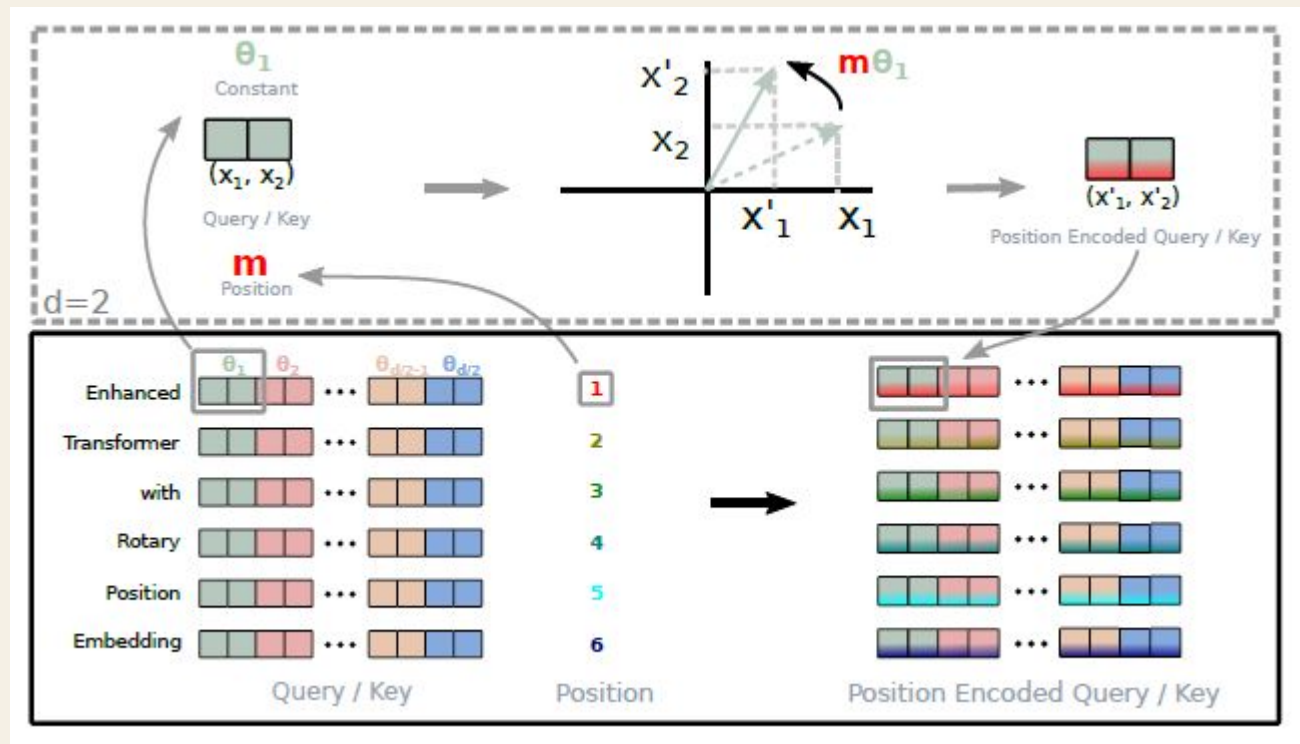
GPT-2 Learned PE $\rightarrow$ LLaMA RoPE (Rotary PE)

- GPT-2 adds learned positional embeddings to token embeddings before the first layer.
- LLaMA injects position by rotating Q and K vectors inside the attention computation.
- This makes attention naturally depend on relative positions, improving long-context generalization.



```
def attention_with_rope(x, position_ids):  
    q = Wq(x)  
    k = Wk(x)  
    v = Wv(x)  
  
    q = apply_rope(q, position_ids)  
    k = apply_rope(k, position_ids)  
  
    scores = (q @ k.transpose(-2, -1)) / math.sqrt(q.size(-1))  
    weights = softmax(scores, dim=-1)  
    return weights @ v
```

Rotary PE



RoPE detail

- RoPE treats each pair of hidden dimensions as a 2D vector and applies a position-dependent rotation.
- Because both Q and K are rotated, the dot product encodes relative displacement between tokens.

```
def apply_rope(x, position_ids, theta_base=10000):  
    # x: (... , head_dim) where head_dim is even  
    freqs = rope_frequencies(head_dim=x.size(-1), base=theta_base) # shape (head_dim/2,)   
    angles = position_ids[..., None] * freqs[None, :] # broadcast to (... , head_dim/2)  
  
    x_even = x[..., 0::2]  
    x_odd = x[..., 1::2]  
  
    x_rot_even = x_even * cos(angles) - x_odd * sin(angles)  
    x_rot_odd = x_even * sin(angles) + x_odd * cos(angles)  
  
    return interleave(x_rot_even, x_rot_odd) # back to (... , head_dim)
```

GPT-2 LayerNorm → LLaMA RMSNorm

- GPT-2 uses LayerNorm, which normalizes both mean and variance.
- LLaMA uses RMSNorm, which normalizes only by root-mean-square magnitude.
- It's cheaper and tends to be more stable in very large models.



```
def rmsnorm(x, weight, eps=1e-6):  
    rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) +  
    eps)  
    return (x / rms) * weight
```

GPT-2 GeLU MLP $\rightarrow$ LLaMA SwiGLU (Gated MLP)

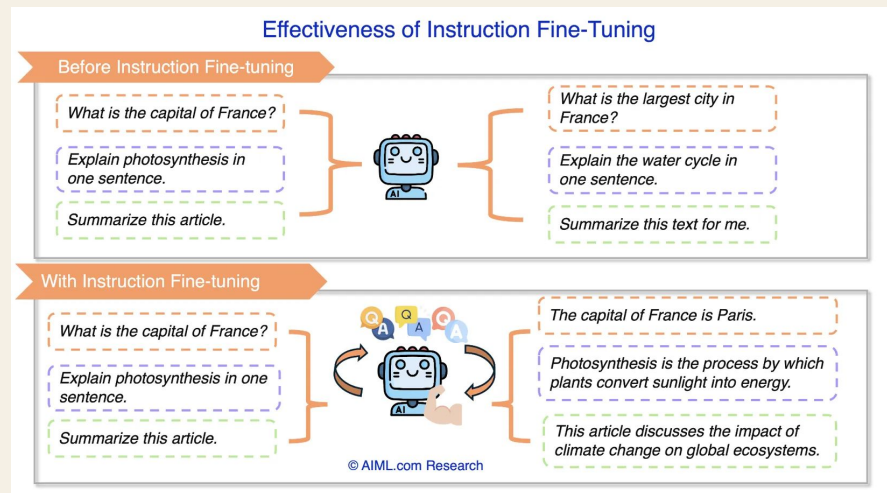
- GPT-2's FFN is a single nonlinearity (e.g., GeLU) followed by a projection.
- LLaMA replaces it with a gated MLP where one projection modulates the other.
- This increases expressiveness and typically improves accuracy per parameter.



```
def silu(x):  
    return x * sigmoid(x)  
  
def swiglu_mlp(x):  
    gate = silu(W_gate(x))    # gating branch  
    up   = W_up(x)           # value branch  
    return W_down(gate * up)
```

LLaMA 2-Chat (Instruction Finetuning)

- A LLM like GPT-2 cannot directly chat with user.
- LLaMA 2-Chat is built by fine-tuning the pretrained LLaMA 2 base model on high-quality instruction-response datasets.
- This training pipeline transforms a next-token predictor into an instruction-following chatbot.



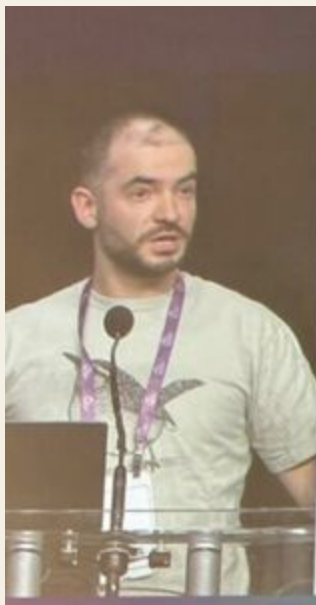


Part IV: Qwen-3

(Alibaba, 2025)

“Llama 4, I think, was such a disappointment. It wasn’t a bad model, but it ... wasn’t amazing at anything.”

Meta CTO Andrew Bosworth



Pre-training as we know it will end

Compute is growing:

- Better hardware
- Better algorithms
- Larger clusters

Data is not growing:

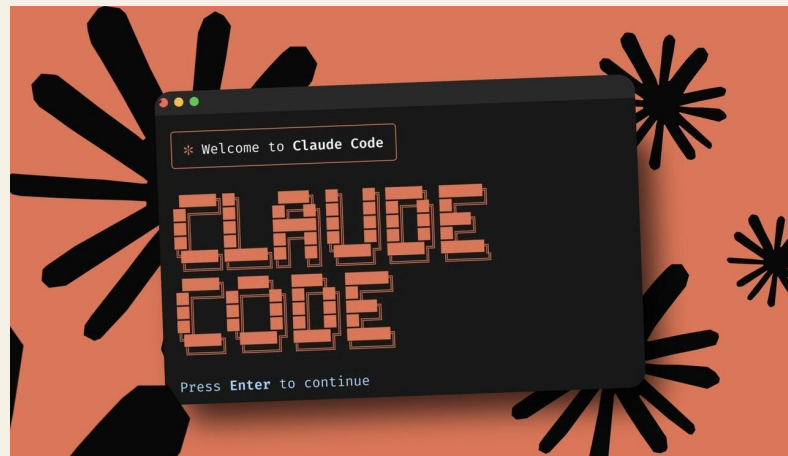
- We have but one internet
- **The fossil fuel of AI**

What comes next?

- "Agents"??
- "Synthetic data"
- Inference time compute ~ O1

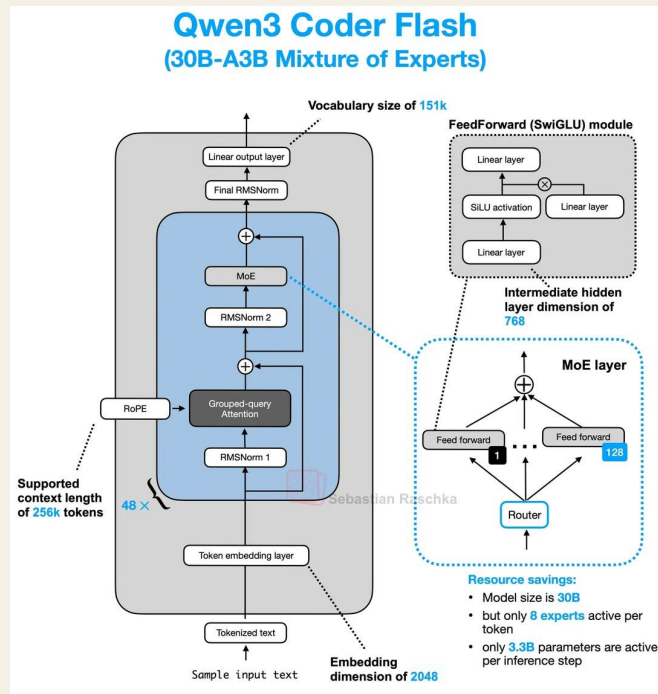
Test-time Scaling and Agents

- Reducing inference cost and latency:
 - LLMs often need to be invoked multiple times to fulfill a single user request.
- Long-context capability:
 - Agent systems tend to accumulate long interaction histories over time.



Qwen-3 Overview

- Qwen-3 builds on the LLaMA-style Transformer, focusing on long-context modeling, efficiency, and reasoning stability.
- More Aggressive Long-Context RoPE Scaling.
- Mixture-of-Experts (MoE) FFN.
- Recent Qwen-3 variants incorporate gated attention, adding nonlinearity directly inside attention.



More Aggressive Long-Context RoPE Scaling

- LLaMA uses standard RoPE with fixed frequency scaling.
- Qwen-3 applies RoPE scaling strategies (e.g. NTK/YaRN-style) to support much longer contexts.
- This allows extrapolation to long sequences without retraining positional embeddings.



```
def apply_scaled_rope(x, position_ids, base=10000, scale_factor=alpha):  
    freqs = rope_frequencies(x.size(-1), base=base)  
    scaled_pos = position_ids * scale_factor  
    return rotate(x, scaled_pos, freqs)
```

Mixture-of-Experts (MoE) FFN

- LLaMA uses a dense feedforward network in every layer.
- Qwen-3 introduces Mixture-of-Experts, activating only a few FFNs per token.
- This scales parameter count dramatically while keeping compute manageable.

```
def moe_ffn(x):  
    scores = router(x) # routing logits  
    topk_ids, topk_w = topk(scores, k=2)  
  
    out = 0  
    for i, w in zip(topk_ids, topk_w):  
        out += w * expert_ffn[i](x)  
  
    return out
```

Gated Attention

- (NeurIPS 2025 Best paper)
- Standard attention is a purely linear weighted sum of values.
- Gated attention multiplies the attention output by a learned, input-dependent gate.
- This adds nonlinearity inside attention and improves stability and long-context behavior.



```
def gated_attention(x, position_ids):  
    q = apply_rope(Wq(x), position_ids)  
    k = apply_rope(Wk(x), position_ids)  
    v = Wv(x)  
  
    attn = softmax(q @ k.transpose(-2, -1))  
    out = attn @ v  
  
    gate = sigmoid(W_gate(x))  
    return out * gate
```



Part V:
Transformers Library
(Huggingface, 2019)

Transformers: Load GPT-2, LLaMA-2, Qwen-3

- HuggingFace Transformers provides a unified API for classic and modern LLMs.
- GPT-2, LLaMA-2, and Qwen-3 are all loaded via `AutoModelForCausalLM`.
- Architectural differences are abstracted behind a consistent interface.



```
from transformers import AutoTokenizer, AutoModelForCausalLM

models = {
    "gpt2": "gpt2",
    "llama2": "meta-llama/Llama-2-7b-hf",
    "qwen3": "Qwen/Qwen3-7B"
}

tokenizers = {}
llms = {}

for name, ckpt in models.items():
    tokenizers[name] = AutoTokenizer.from_pretrained(ckpt)
    llms[name] = AutoModelForCausalLM.from_pretrained(ckpt)
```

Transformers: Same Generation Code for All LLMs

- Transformers provides a standardized `generate()` interface for decoder-only LLMs.
- Sampling strategies like temperature and top-p control output diversity.
- KV caching and causal masking are handled automatically during decoding.

```
def generate_text(model, tokenizer, prompt):  
    inputs = tokenizer(prompt, return_tensors="pt")  
    outputs = model.generate(  
        **inputs,  
        max_new_tokens=50,  
        do_sample=True,  
        temperature=0.7  
    )  
    return tokenizer.decode(outputs[0],  
        skip_special_tokens=True)  
text = generate_text(llms["qwen3"], tokenizers["qwen3"],  
    "Explain LoRA in one sentence:")  
print(text)
```

Transformers: Load GPT-2, LLaMA-2, Qwen-3

- Transformers provides a high-level training pipeline through the Trainer API.
- It handles batching, checkpointing, evaluation, and optimizer scheduling.
- This makes fine-tuning large Transformer models accessible with minimal boilerplate.

```
from transformers import Trainer, TrainingArguments

args = TrainingArguments(
    output_dir="out",
    per_device_train_batch_size=2,
    learning_rate=2e-5,
    num_train_epochs=1,
)

trainer = Trainer(
    model=llms["llama2"],
    args=args,
    train_dataset=train_data,
)

trainer.train()
```

Thank you!

Any questions?